

Preuves assistées par ordinateur pour les équations différentielles et les systèmes dynamiques

Damien Bourgouin - Zakaria Otmane

2025/2026



UFR mathématiques, Université de Rennes

Département des Mathématiques, IRMAR

Projet de recherche : Introduction théorique à la notion de preuve assistée par
ordinateur

Superviseur : Professeur Vincent Duchene

Table des matières

Introduction	1
1 Arithmétique d'intervalles	2
1.1 Motivations	2
1.2 Formalisme mathématique	2
1.3 Un exemple illustratif	4
2 Résultats théoriques pour la recherche de zéros	4
2.1 Rappel de la méthode de Newton et du théorème du point fixe contractant	5
2.2 Théorèmes de Newton–Kantorovich	6
3 Applications	9
3.1 Application à l'équation de Allen–Cahn	9
Conclusion	15
Annexes	
Notations	
Quelques preuves complémentaires	
Codes Python	

Introduction

L'étude numérique des zéros de fonctions et d'opérateurs constitue un enjeu central en analyse appliquée et en calcul scientifique. De nombreux problèmes issus de la physique ou de l'ingénierie se ramènent à la résolution d'équations non linéaires pour lesquelles aucune solution explicite n'est accessible. Si les méthodes numériques classiques, comme la méthode de Newton, permettent d'obtenir des approximations de plus en plus précises, elles ne constituent pas, en soi, une preuve mathématique de l'existence d'une solution exacte.

L'objectif de ce travail est de présenter la méthodologie des **preuves assistées par ordinateur**. Cette approche vise à combler le fossé entre le calcul numérique et la rigueur analytique en utilisant l'ordinateur non plus seulement comme un outil d'approximation, mais comme un instrument de démonstration. Cette démarche repose sur deux piliers fondamentaux que nous explorerons :

- **L'arithmétique d'intervalles**, qui permet de gérer rigoureusement les erreurs d'arrondi et les imprécisions inhérentes au calcul sur ordinateur, garantissant que le résultat d'une opération est inclus dans un intervalle contrôlé.
- **Le théorème de Newton–Kantorovich**, qui fournit des conditions suffisantes pour garantir l'existence et l'unicité d'une solution exacte au voisinage d'une solution numérique approximative.

Nous illustrerons ces concepts par l'étude de l'équation d'Allen–Cahn. En utilisant une décomposition en série de Fourier, nous transformerons ce problème différentiel en un système d'équations algébriques dans un espace de suites de Banach (ℓ_ν^1). Nous montrerons alors comment la construction d'un inverse approché, combinée à des calculs rigoureux sur une partie finie du système, permet de valider l'existence d'une solution dans un espace de dimension infinie.

Ce rapport est structuré de la manière suivante :

- **Chapitre 1** : Présentation de l'arithmétique d'intervalles. Nous abordons les motivations liées au contrôle des erreurs d'arrondi, le formalisme mathématique des extensions d'intervalles et un exemple illustratif montrant l'efficacité de cette approche pour garantir des inclusions rigoureuses.
- **Chapitre 2** : Résultats théoriques pour la recherche de zéros. Ce chapitre rappelle les fondements de la méthode de Newton et du théorème du point fixe contractant, avant de détailler les théorèmes de Newton–Kantorovich qui permettent de valider mathématiquement l'existence d'une solution unique.
- **Chapitre 3** : Application à l'équation d'Allen–Cahn. Nous mettons en pratique les outils précédents en transformant une équation différentielle en un système algébrique infini via les séries de Fourier, puis nous utilisons l'algorithme de Newton rigoureux pour valider une solution dans l'espace des suites ℓ_ν^1 .

Les codes informatiques (Python/Arb) correspondant aux algorithmes de validation évoqués dans ce rapport, ainsi que certaines démonstrations mathématiques jugées complémentaires, sont regroupés en annexes pour consultation.

1 Arithmétique d'intervalles

Nous commençons cette section par introduire la notion d'arithmétique d'intervalles. Il s'agit d'une méthode, à la croisée des mathématiques et de l'informatique, qui consiste à manipuler des intervalles plutôt que des nombres ponctuels (tels que les entiers ou les nombres flottants), dans le but d'obtenir des résultats numériques rigoureux.

1.1 Motivations

Les principales motivations associées à ce concept résident dans la nécessité de maîtriser et de borner rigoureusement les erreurs d'arrondi, ainsi que les erreurs numériques introduites par les méthodes algorithmiques, afin de concevoir des procédures de calcul garantissant des résultats fiables.

En effet, certains nombres réels, tels que e , $\sqrt{2}$ ou $\frac{1}{3}$ ne peuvent être représentés exactement en arithmétique machine. Par ailleurs, les opérations effectuées sur de telles quantités sont nécessairement affectées par des erreurs d'approximation liées à leur représentation finie.

Par exemple, Python utilise l'arithmétique en double précision (norme IEEE 754¹). Le nombre $\sqrt{2}$ n'est pas représentable exactement en base binaire ; toute manipulation numérique repose donc sur des approximations.

On peut illustrer ce phénomène à l'aide du code Python suivant :

```
import math
math.sqrt(2)**2 == 2
```

qui renvoie `False`.

En effet, une telle évaluation donne typiquement 2.0000000000000004, ce qui met en évidence l'erreur d'arrondi inhérente à la représentation en virgule flottante.

L'arithmétique d'intervalles permet alors de fournir un cadre rigoureux dans lequel les résultats des opérations sont systématiquement encadrés, en garantissant que la valeur exacte appartient toujours à l'intervalle obtenu.

1.2 Formalisme mathématique

Pour formaliser cette approche, on considère $\mathbb{F}$ l'ensemble des nombres représentables, c'est-à-dire l'ensemble des nombres réels qu'un ordinateur peut représenter à une précision donnée. On introduit les opérateurs d'arrondi inférieur et supérieur, notés respectivement $\nabla, \triangle : \mathbb{R} \rightarrow \mathbb{F}$.

1. Il s'agit d'un standard international qui définit la représentation et la manipulation des nombres réels en virgule flottante par les ordinateurs. Un nombre y est codé sous la forme :

$$\text{nombre} = (-1)^{\text{signe}} \times \text{mantisse} \times 2^{\text{exposant}}.$$

Autrement dit, pour tout $x \in \mathbb{R}$,

$$\nabla(x) := \max\{y \in \mathbb{F} \mid y \leq x\} \quad \text{et} \quad \Delta(x) := \min\{y \in \mathbb{F} \mid y \geq x\}.$$

On a alors :

$$\nabla(x) \leq x \leq \Delta(x),$$

avec égalité si et seulement si $x \in \mathbb{F}$. Ainsi, un réel x n'est plus identifié à une valeur unique, mais à un encadrement garanti.

Plus précisément, on associe à x l'intervalle :

$$[x] := [\nabla(x), \Delta(x)],$$

qui constitue le plus petit intervalle représentable contenant x .

Un point essentiel de cette construction est qu'elle garantit, par définition, l'appartenance de x à l'intervalle $[x]$, même lorsque $x \notin \mathbb{F}$.

Dans ce cadre, les opérations arithmétiques usuelles sont étendues aux intervalles. L'idée consiste à combiner une définition ensembliste naturelle avec un mécanisme d'arrondi dirigé, de manière à préserver la propriété d'encadrement. Ainsi, pour deux intervalles $I = [a, b]$ et $J = [c, d]$, où $(a, b, c, d) \in \mathbb{F}^4$, on définit leur somme par :

$$I + J := [\nabla(a + c), \Delta(b + d)].$$

Si x et y sont remplacés par leurs intervalles associés $[x]$ et $[y]$, alors $[x] + [y]$ contient la valeur exacte $x + y$, ce qui garantit la validité de l'encadrement au niveau de l'opération.

De manière analogue, les opérations de soustraction, de multiplication et de division peuvent être étendues au cadre des intervalles.

On définit par exemple :

$$I - J := [\nabla(a - d), \Delta(b - c)].$$

La multiplication repose sur l'évaluation de toutes les combinaisons possibles des bornes :

$$I \times J := [\nabla(\min(ac, ad, bc, bd)), \Delta(\max(ac, ad, bc, bd))].$$

Pour la division, sous l'hypothèse que $0 \notin J$, on introduit l'intervalle des inverses :

$$I \div J := I \times \left[\nabla\left(\frac{1}{d}\right), \Delta\left(\frac{1}{c}\right) \right].$$

Ce cadre peut également être étendu à des fonctions plus générales telles que $\exp$ ou $\cos$. L'objectif est alors, pour toute fonction f et tout intervalle représentable I , de construire un intervalle $f(I)$ contenant l'ensemble des valeurs $\{f(x) \mid x \in I\}$. Cette propriété garantit que chaque évaluation numérique reste rigoureusement encadrée.

1.3 Un exemple illustratif

Soit $f : \mathbb{R}_+^* \rightarrow \mathbb{R}$ définie par

$$f(x) := \cos(\ln(x)) + \frac{1}{x} + x^3 - 23,$$

représentée sur la figure 1 ci-dessous.

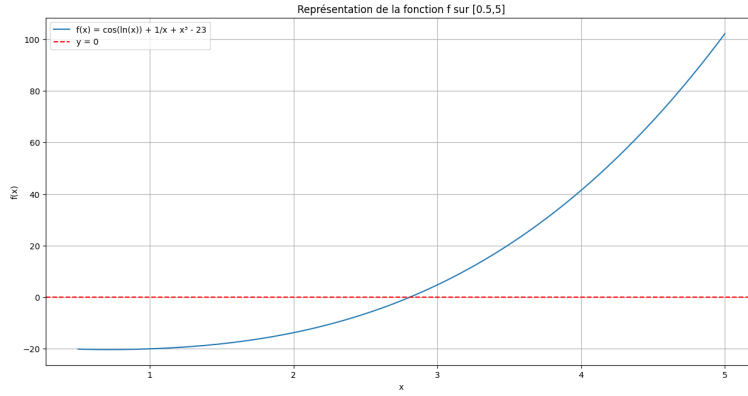


FIGURE 1 – Graphe de la fonction f définie précédemment

On s'intéresse à la valeur de $f(1)$. Il est clair que $f(1) = -20$ mais dans un cadre numérique classique, cette évaluation est sujette aux erreurs d'arrondi inhérentes à la représentation en virgule flottante.

Afin d'obtenir une évaluation rigoureuse, on utilise la bibliothèque **flint** (via son interface **arb**), qui implémente l'arithmétique d'intervalles en effectuant des arrondis dirigés de manière à garantir l'encadrement des résultats. On obtient alors

$$f(1) \in [-20.0000000000000003553, -19.9999999999999996447],$$

ce qui constitue un encadrement très précis de la valeur exacte -20 .

Cependant, cette méthode présente aussi des limites. Par exemple, si on pose $I = [0, 1]$, on a :

$$I - I = [-1, 1],$$

alors que pour tout $x \in I$, $x - x = 0$. L'intervalle obtenu est donc une surestimation stricte de la valeur exacte.

2 Résultats théoriques pour la recherche de zéros

L'intérêt de l'arithmétique d'intervalles ne se limite pas à l'encadrement de valeurs numériques. Elle constitue également un outil fondamental pour la va-

lvalidation rigoureuse de résultats d'existence, en particulier dans le cadre de la recherche de zéros de fonctions. Dans la suite de ce travail, nous mettrons en évidence comment ces techniques peuvent être combinées avec des méthodes itératives, telles que la méthode de Newton, afin d'obtenir des résultats garantis. Nous commençons par rappeler la méthode de Newton ainsi que le théorème du point fixe contractant.

2.1 Rappel de la méthode de Newton et du théorème du point fixe contractant

En analyse numérique, la méthode de Newton est un algorithme efficace permettant de déterminer numériquement une approximation précise d'un zéro d'une fonction réelle d'une variable réelle.

Précisément, pour $f : \mathbb{R} \rightarrow \mathbb{R}$ une fonction dérivable. On cherche à déterminer $x^* \in \mathbb{R}$ tel que $f(x^*) = 0$.

On se donne une valeur initiale $x_0 \in \mathbb{R}$, choisie de préférence proche de x^* , puis on approche f par son développement de Taylor à l'ordre 1 au voisinage de x_0 :

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0).$$

On considère alors l'équation :

$$f(x_0) + f'(x_0)(x - x_0) = 0,$$

dont on note x_1 la solution. Cette dernière fournit en général une meilleure approximation de x^* que x_0 .

On itère ensuite ce procédé pour construire une suite $(x_k)_{k \geq 0}$ définie par la relation de récurrence :

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}. \quad (1)$$

On peut, dans certains cas, garantir la convergence de la méthode par l'intermédiaire du théorème du point fixe.

Théorème 1 : Soit $(E, \|\cdot\|)$ un espace de Banach, et soit $\varphi : E \rightarrow E$ une application contractante, c'est-à-dire qu'il existe $L \in [0, 1[$ tel que, pour tout $(x, y) \in E^2$,

$$\|\varphi(x) - \varphi(y)\| \leq L\|x - y\|.$$

Alors φ admet un unique point fixe x^* . De plus, toute suite $(x_k)_{k \geq 0}$ définie par $x_0 \in E$ et

$$x_{k+1} = \varphi(x_k),$$

2. La méthode se généralise pour $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ différentiable, de jacobienne DF . Sous réserve que $DF(x_k)$ soit inversible, l'algorithme s'écrit :

$$x_{k+1} = x_k - DF(x_k)^{-1}F(x_k).$$

converge vers x^* . Enfin, on dispose de l'estimation suivante :

$$\|x_{k+1} - x^*\| \leq \frac{L}{1-L} \|x_{k+1} - x_k\|. \quad (2)$$

En effet, en posant :

$$\varphi(x) := x - \frac{f(x)}{f'(x)},$$

les itérations de la méthode de Newton s'écrivent :

$$x_{k+1} = \varphi(x_k).$$

Ainsi, sous réserve que φ soit contractante, le théorème 1 garantit que x^* est l'unique point fixe de φ , et que la suite $(x_k)_{k \geq 0}$ converge vers x^* . De plus, l'inégalité (2) fournit un critère d'arrêt pour la méthode. On peut en effet poser, pour $\varepsilon > 0$, le critère :

$$\|x_{k+1} - x_k\| < \varepsilon.$$

Le paramètre ε représente une majoration de l'erreur d'approximation associée à la première itération de la méthode de Newton. Cette étape correspond à la construction d'une solution approchée, qui sert ensuite de point de référence pour l'analyse.

Il convient de distinguer deux niveaux dans cette approche :

- L'application de la méthode de Newton, qui fournit une approximation numérique d'une solution.
- L'utilisation, dans la prochaine section, du théorème de Newton-Kantorovich qui permet de valider rigoureusement l'existence et l'unicité d'une solution exacte dans un voisinage de cette approximation.

Nous concluons cette section en soulignant que la méthode présentée est formulée en dimension finie. La section suivante sera consacrée à son extension au cadre des espaces de dimension infinie.

2.2 Théorèmes de Newton–Kantorovich

Dans cette partie, nous travaillerons dans un espace de Banach, noté $\mathcal{X}$. Pour $x \in \mathcal{X}$ et $r \geq 0$, nous noterons $\overline{B(x, r)}$ la boule fermée de centre x et de rayon r . Nous commençons par énoncer un théorème de point fixe.

Théorème 2 : *Soit $\mathcal{X}$ un espace de Banach, $\bar{x} \in \mathcal{X}$ et $T : \mathcal{X} \rightarrow \mathcal{X}$ une application de classe $\mathcal{C}^1$. Supposons qu'il existe $r^* > 0$ et des constantes $Y, Z_1, Z_2 \geq 0$ telles que :*

$$\|T(\bar{x}) - \bar{x}\| \leq Y,$$

$$\|DT(\bar{x})\| \leq Z_1,$$

$$\|DT(x) - DT(\bar{x})\| \leq Z_2 \|x - \bar{x}\|, \quad \forall x \in \overline{B(\bar{x}, r^*)}.$$

S'il existe $r \in [0, r^*]$ tel que :

$$Y + Z_1 r + \frac{1}{2} Z_2 r^2 \leq r, \quad \text{et} \quad Z_1 + Z_2 r < 1,$$

alors T admet un unique point fixe x^* dans $\overline{B(\bar{x}, r)}$

Preuve : Soient $r \in [0, r^*]$ et $x \in \overline{B(\bar{x}, r)}$, on utilise le théorème de Taylor

$$\begin{aligned} T(x) - \bar{x} &= T(\bar{x}) - \bar{x} + \int_0^1 DT(\bar{x} + t(x - \bar{x}))(x - \bar{x}) dt \\ &= T(\bar{x}) - \bar{x} + DT(\bar{x})(x - \bar{x}) \\ &\quad + \int_0^1 (DT(\bar{x} + t(x - \bar{x})) - DT(\bar{x}))(x - \bar{x}) dt \end{aligned}$$

On obtient donc :

$$\begin{aligned} \|T(x) - \bar{x}\| &\leq Y + Z_1 \|x - \bar{x}\| + Z_2 \|x - \bar{x}\|^2 \int_0^1 t dt \\ &\leq Y + Z_1 r + \frac{1}{2} Z_2 r^2 \\ &\leq 1 \end{aligned}$$

ce qui assure que T envoie $\overline{B(\bar{x}, r)}$ dans lui même. En outre, par l'inégalité triangulaire :

$$\|DT(x)\| \leq \|DT(\bar{x})\| + \|DT(x) - DT(\bar{x})\| \leq Z_1 + Z_2 r < 1$$

ce qui assure que T est contractante sur $\overline{B(\bar{x}, r)}$. L'existence et l'unicité d'un point fixe de T dans $\overline{B(\bar{x}, r)}$ découle alors du théorème du point fixe de Banach. $\square$

Le théorème 2 sera crucial pour la mise en œuvre de preuves assistées par ordinateur. Son utilisation intervient de la manière suivante. Soient $\mathcal{X}$ et $\mathcal{Y}$ deux espaces de Banach et $F : \mathcal{X} \rightarrow \mathcal{Y}$ une application de classe C^1 . On cherche un élément $x^* \in \mathcal{X}$ tel que $F(x^*) = 0$.

On considère un élément $\bar{x} \in \mathcal{X}$, qui constitue une approximation numérique de x^* , typiquement obtenue par une méthode numérique (de type Newton par exemple). On introduit alors l'application T définie par $T(x) := x - DF(\bar{x})^{-1}F(x)$. Si F admet effectivement un zéro localement unique au voisinage de $\bar{x}$, on s'attend à ce que T soit contractante sur un voisinage de $\bar{x}$.

Notons que plusieurs difficultés apparaissent. D'une part, l'opérateur $DF(\bar{x})$ peut être difficile à manipuler explicitement. D'autre part, son inversibilité n'est pas garantie a priori. Par conséquent, il est souvent préférable d'introduire un opérateur A , censé approcher $DF(\bar{x})^{-1}$, afin de contourner ces difficultés. Nous énonçons alors le théorème suivant :

Théorème 3 : Soient $\mathcal{X}$ et $\mathcal{Y}$ deux espaces de Banach $F : \mathcal{X} \rightarrow \mathcal{Y}$ une application de classe C^1 et $A : \mathcal{Y} \rightarrow \mathcal{X}$ une application linéaire injective. On suppose qu'il existe un rayon $r^* \in]0, +\infty]$ et des constantes $Y, Z_1, Z_2 \geq 0$ telles que :

$$\begin{aligned} \|AF(\bar{x})\|_{\mathcal{X}} &\leq Y \\ \|I - ADF(\bar{x})\|_{\mathcal{L}(\mathcal{X})} &\leq Z_1 \\ \|A(DF(x) - DF(\bar{x}))\|_{\mathcal{L}(\mathcal{X})} &\leq Z_2\|x - \bar{x}\|_{\mathcal{X}}, \quad \forall x \in \overline{B(\bar{x}, r^*)}. \end{aligned}$$

S'il existe $r \in [0, r^*]$ tel que :

$$Y + Z_1r + \frac{1}{2}Z_2r^2 \leq r, \quad \text{et} \quad Z_1 + Z_2r < 1,$$

alors F admet un unique zéro x^* dans $\overline{B(\bar{x}, r)}$.

Preuve : Il suffit d'appliquer le théorème 2 à $T : x \mapsto x - AF(x)$. □

Le rôle des constantes intervenant dans le théorème 3 mérite une interprétation précise.

La quantité Y mesure l'erreur associée à l'approximation numérique $\bar{x}$, en évaluant la taille de $F(\bar{x})$ après préconditionnement par l'opérateur A . La constante Z_1 quantifie la qualité de l'approximation de l'inverse de la différentielle et enfin, la constante Z_2 contrôle la non-linéarité de l'application F au voisinage de $\bar{x}$, en fournissant une borne sur les variations de sa différentielle.

Ainsi, ces trois paramètres permettent d'isoler les différentes sources d'erreur : l'approximation initiale, l'approximation de l'inverse et les effets non linéaires. L'existence d'un rayon $r > 0$ satisfaisant les hypothèses du théorème 3 dépend directement des valeurs des constantes Y , Z_1 et Z_2 . En particulier, la condition $Z_1 < 1$ est nécessaire, afin que la partie linéaire de l'estimation demeure contractante. Plus précisément, si Y est suffisamment petit, et si les effets non linéaires mesurés par Z_2 restent contrôlés, alors il existe un rayon r tel que les inégalités soient satisfaites.

Ce résultat met en évidence un principe fondamental : une approximation numérique suffisamment précise d'un zéro, combinée à une bonne approximation de l'inverse de la différentielle, permet d'obtenir une preuve rigoureuse d'existence et d'unicité d'une solution exacte.

En ce sens, le théorème 3 constitue un résultat de nature *a posteriori* : à partir d'une approximation numérique calculée, on déduit un énoncé mathématique rigoureux garantissant l'existence d'une solution véritable à proximité.

En pratique, l'application du théorème 3 repose sur deux étapes essentielles : reformuler le problème dans un espace de Banach adapté et établir des estimations explicites des constantes Y , Z_1 et Z_2 vérifiant les hypothèses du théorème.

3 Applications

Les résultats des sections section 1 et section 2 vont maintenant être appliqués à la résolution d'une équation différentielle, l'équation d'Allen–Cahn. L'équation d'Allen–Cahn est un modèle fondamental de la physique statistique et de la science des matériaux, introduite initialement pour décrire la séparation de phase dans les alliages binaires. La méthodologie que nous allons présenter s'étend à des modèles plus complexes, comme l'équation de Swift-Hohenberg. Une application détaillée de ces techniques à ce système est présentée dans [1, Section 1.3.1].

3.1 Application à l'équation de Allen–Cahn

Nous considérons l'équation d'Allen–Cahn stationnaire sur l'intervalle $]0, \pi[$, définie par :

$$-u'' + u - u^3 = 0, \quad (3)$$

associée aux conditions aux limites de Neumann :

$$u'(0) = u'(\pi) = 0.$$

Compte tenu des conditions de Neumann, nous cherchons la solution u sous la forme d'un développement en série de Fourier :

$$u(x) = u_0 + 2 \sum_{n=1}^{+\infty} u_n \cos(nx). \quad (4)$$

Par cette décomposition, la fonction u est identifiée à la suite de ses coefficients de Fourier et on écrit abusivement $u = (u_n)_{n \in \mathbb{N}}$. Nous supposons que $(u_n)_{n \in \mathbb{N}} \in \ell_\nu^1$.

En substituant l'expression (4) dans l'équation (3) et en utilisant les propriétés du produit de convolution (voir le théorème A1 en annexe), nous avons :

$$-u''(x) + u(x) - u^3(x) = u_0 - (u * u * u)_0 + 2 \sum_{n=1}^{+\infty} ((n^2 + 1)u_n - (u * u * u)_n) \cos(nx).$$

L'équation (3) est alors satisfaite si et seulement si chaque coefficient de la série est nul, ce qui nous ramène à la résolution du système infini :

$$(n^2 + 1)u_n - (u * u * u)_n = 0, \quad \forall n \in \mathbb{N}.$$

Pour traiter ce problème de manière globale, nous introduisons l'opérateur F défini sur ℓ_ν^1 par :

$$\begin{aligned} F : \ell_\nu^1 &\longrightarrow \ell_\nu^1 \\ u &\longmapsto (F_n(u))_{n \in \mathbb{N}}, \end{aligned}$$

où chaque composante est donnée par $F_n(u) := (n^2 + 1)u_n - (u * u * u)_n$. Trouver une solution à (3) revient donc à chercher les zéros de F dans ℓ_ν^1 :

$$F(u) = 0. \quad (5)$$

L'opérateur F se décompose naturellement en une partie linéaire L et une partie non linéaire $\mathcal{M}$:

$$F(u) = Lu - \mathcal{M}(u).$$

Ici, L est l'opérateur diagonal (matrice diagonale infinie) agissant sur les coefficients par :

$$(Lu)_n := (n^2 + 1)u_n, \quad \forall n \in \mathbb{N},$$

et le terme non linéaire est défini par la triple convolution :

$$\mathcal{M}(u) := u * u * u.$$

Enfin, pour appliquer la méthode de Newton-Kantorovich, nous déterminons la différentielle de F en un point u . Elle est représentée par la matrice jacobienne infinie $DF(u) = \left(\frac{\partial F_n(u)}{\partial u_j} \right)_{n,j \in \mathbb{N}}$, dont les coefficients sont :

$$DF(u)_{n,j} = (n^2 + 1)\delta_{jn} - 3(u * u * e^{(j)})_n,$$

où δ_{jn} est le symbole de Kronecker et $e^{(j)}$ désigne le j -ème vecteur de la base canonique, défini par $e_k^{(j)} = 1$ si $k = j$ et 0 sinon.

Pour obtenir une solution numérique approchée $\bar{u}$, nous fixons un entier $N \in \mathbb{N}$ afin de définir la décomposition en somme directe suivante :

$$\ell_\nu^1 = X_N \oplus X_\infty,$$

où X_N est le sous-espace de dimension finie défini par :

$$X_N := \{u \in \ell_\nu^1 \mid u_n = 0 \text{ pour } n > N\},$$

et X_∞ est le sous-espace de dimension infinie défini par :

$$X_\infty := \{u \in \ell_\nu^1 \mid u_n = 0 \text{ pour } 0 \leq n \leq N\}.$$

On note également $\Pi_N : \ell_\nu^1 \rightarrow X_N$ l'opérateur de projection sur X_N . Nous introduisons l'opérateur F^N défini sur X_N par :

$$\begin{aligned} F^N : \ell_\nu^1 &\longrightarrow \ell_\nu^1 \\ u &\longmapsto \Pi_N F \Pi_N(u), \end{aligned}$$

Pour $u \in X_N$, nous ramenons le problème de dimension infinie (5) à un système fini de $N + 1$ équations à $N + 1$ inconnues :

$$F^N(u) = 0.$$

Ce système est résolu de manière approchée par une méthode de Newton, ce qui nous permet de définir notre solution numérique $\bar{u} \in X_N$.

Ensuite pour le calcul de la matrice A , notons que :

$$DF(\bar{u}) = L - D\mathcal{M}(\bar{u}),$$

avec :

$$(D\mathcal{M}(\bar{u})h)_n = 3(\bar{u} * \bar{u} * h)_n.$$

L'opérateur L est diagonal et inversible sur ℓ_ν^1 , avec :

$$(L^{-1}h)_n = \frac{h_n}{n^2 + 1}.$$

De plus, lorsque $n \rightarrow +\infty$, la partie diagonale domine le terme de convolution, de sorte que :

$$DF(\bar{u}) \approx L.$$

Ceci suggère d'utiliser L^{-1} comme approximation de l'inverse de $DF(\bar{u})$ sur la queue spectrale (c'est à dire lorsque $n > N$).

L'action de $DF(\bar{u})$ sur X_N dans X_N est représentée par la matrice finie :

$$J := DF^{(N)}(\bar{u}) = \Pi_N DF(\bar{u}) \Pi_N.$$

On calcule numériquement une approximation de son inverse, notée :

$$R \approx J^{-1}.$$

L'inversibilité de J est certifiée par arithmétique des intervalles, en vérifiant que l'intervalle obtenu pour $\det(J)$ ne contient pas 0. L'opérateur approché $A : \ell_\nu^1 \rightarrow \ell_\nu^1$ est alors défini par blocs :

$$A = \begin{pmatrix} R & 0 \\ 0 & L^{-1} \end{pmatrix}.$$

Autrement dit, pour $h = (h_n)_{n \geq 0} \in \ell_\nu^1$,

$$(Ah)_n = \begin{cases} (Rh^{(N)})_n, & 0 \leq n \leq N, \\ \frac{h_n}{n^2 + 1}, & n > N, \end{cases}$$

où

$$h^{(N)} = (h_0, \dots, h_N).$$

Cette construction présente deux avantages fondamentaux.

- Sur les basses fréquences, c'est à dire $n \leq N$, l'opérateur R fournit une approximation précise de l'inverse de $DF(\bar{u})$.
- Sur les hautes fréquences, c'est à dire $n > N$, l'opérateur diagonal L^{-1} capture le comportement asymptotique dominant de $DF(\bar{u})^{-1}$, ce qui permet d'obtenir des estimations analytiques explicites.

Ainsi, l'opérateur A combine un calcul numérique fini avec un contrôle analytique de la queue infinidimensionnelle, ce qui constitue le principe fondamental des méthodes de validation assistée par ordinateur de ce type.

Le calcul de Y se fera simplement avec la formule :

$$Y := \|AF(\bar{u})\|_\nu.$$

En effet, $\bar{u} \in X_N$, de ce fait $AF(\bar{u})$ sera un élément de taille finie (précisément de taille $3N + 1$), ce qui fera de Y un élément calculable par l'ordinateur. Nous allons à présent décomposer l'estimation de Z_1 en deux parties :

$$Z_1 = \max\left(Z_1^{fini}, Z_1^{queue}\right).$$

On pose :

$$B := I - ADF(\bar{u})$$

On étudie la restriction de B à l'espace fini X_N . Avec une telle restriction B peut être représentée de manière finie comme une matrice $(3N + 1) \times (N + 1)$. Nous devons maintenant calculer une borne de la norme opérateur de B sur ℓ_ν^1 .

Dans notre décomposition de Fourier, la norme $\|u\|_\nu$ introduit les poids :

$$\omega_0 = 1, \quad \omega_n = 2\nu^n \quad (n \geq 1).$$

De par le lemme A3 de l'annexe, la norme opérateur induite d'une matrice $M = (M_{ij})$ est alors donnée par :

$$\|M\|_{\mathcal{L}(\ell_\nu^1)} = \sup_{j \geq 0} \frac{1}{\omega_j} \sum_{i=0}^{+\infty} |M_{ij}| \omega_i.$$

On obtient donc :

$$Z_1^{fini} = \|B\|_{\mathcal{L}(\ell_\nu^1)} = \max_{0 \leq j \leq N} \frac{1}{\omega_j} \sum_{i=0}^{3N} |B_{ij}| \omega_i.$$

Cette quantité est calculée explicitement et numériquement à l'aide de l'arithmétique des intervalles. Considérons maintenant les hautes fréquences $n > N$. Sur cette partie, l'opérateur approché est défini par :

$$(Ah)_n = \frac{h_n}{n^2 + 1}.$$

Comme :

$$DF(\bar{u}) = L - DM(\bar{u}),$$

on obtient :

$$B = I - L^{-1}(L - DM(\bar{u})) = L^{-1}DM(\bar{u}).$$

Ainsi, pour $n > N$:

$$(Bh)_n = \frac{1}{n^2 + 1} (DM(\bar{u})h)_n.$$

Or :

$$(DM(\bar{u})h)_n = 3(\bar{u} * \bar{u} * h)_n.$$

En utilisant le Théorème A2 en annexe, on obtient :

$$\|D\mathcal{M}(\bar{u})h\|_\nu \leq 3\|\bar{u}\|_\nu^2 \|h\|_\nu.$$

De plus, comme $n > N$, on en déduit :

$$\|L^{-1}D\mathcal{M}(\bar{u})\|_{\mathcal{L}(\ell_\nu^1)} \leq \frac{3\|\bar{u}\|_\nu^2}{(N+1)^2 + 1}.$$

On définit alors :

$$Z_1^{queue} = \frac{3\|\bar{u}\|_\nu^2}{(N+1)^2 + 1}.$$

Finalement :

$$Z_1 = \max\left(Z_1^{fini}, Z_1^{queue}\right).$$

Cette constante fournit une borne rigoureuse de :

$$\|I - ADF(\bar{u})\|_{\mathcal{L}(\ell_\nu^1)}.$$

Nous cherchons maintenant Z_2 . Soit $u \in \overline{B(\bar{u}, r^*)}$. On a alors :

$$DF(u)h - DF(\bar{u})h = -3(u * u * h) + 3(\bar{u} * \bar{u} * h).$$

Par conséquent :

$$DF(u)h - DF(\bar{u})h = -3(((u + \bar{u}) * (u - \bar{u})) * h).$$

En utilisant l'inégalité de convolution dans ℓ_ν^1 , on obtient :

$$\|DF(u)h - DF(\bar{u})h\|_\nu \leq 3\|u + \bar{u}\|_\nu \|u - \bar{u}\|_\nu \|h\|_\nu.$$

Ainsi :

$$\|DF(u)h - DF(\bar{u})h\|_\nu \leq 3(2\|\bar{u}\|_\nu + r^*)\|u - \bar{u}\|_\nu \|h\|_\nu.$$

On en déduit :

$$\|DF(u) - DF(\bar{u})\|_{\mathcal{L}(\ell_\nu^1)} \leq 3(2\|\bar{u}\|_\nu + r^*)\|u - \bar{u}\|_\nu.$$

En appliquant maintenant l'opérateur A , on obtient finalement :

$$\|A(DF(u) - DF(\bar{u}))\|_{\mathcal{L}(\ell_\nu^1)} \leq 3\|A\|_{\mathcal{L}(\ell_\nu^1, \ell_\nu^1)}(2\|\bar{u}\|_\nu + r^*)\|u - \bar{u}\|_\nu.$$

On définit donc :

$$Z_2 = 3\|A\|_{\mathcal{L}(\ell_\nu^1)}(2\|\bar{u}\|_\nu + r^*).$$

Avec :

$$\|A\|_{\mathcal{L}(\ell_\nu^1)} = \max\left(\|R\|_{\mathcal{L}(\ell_\nu^1)}, \frac{1}{(N+1)^2 + 1}\right).$$

L'implémentation numérique repose ensuite sur la bibliothèque **flint** (via l'interface Python **python-flint**), qui permet de manipuler les types **arb** et **arb_mat**. Ces types représentent des nombres réels sous forme d'intervalles, garantissant que chaque opération arithmétique élémentaire est rigoureusement encadrée par arithmétique des intervalles.

Convolution étendue : La fonction `convolution_extended` calcule le produit de Cauchy jusqu'au mode $3N$, capturant l'intégralité du terme u^3 pour une norme ℓ_ν^1 rigoureuse.

Opérateur A : La Jacobienne `DF_matrix_extended` est de taille $(3N + 1) \times (N + 1)$. L'inverse A combine une matrice numérique R (modes bas) et l'opérateur analytique L^{-1} (modes hauts).

Bornes de validation : Le résidu Y et la contraction Z_1 sont évalués sur l'espace étendu X_{3N} . Z_2 est estimé via la norme de l'opérateur A pour borner la courbure de la non-linéarité.

Certification : Le script valide l'existence et l'unicité en vérifiant par arithmétique d'intervalles (`arb`) que le polynôme de rayon $p(r)$ est négatif et que la condition de contraction $Z_1 + Z_2 r < 1$ est satisfaite.

Nous allons appliquer nos codes avec $\nu := 1.1$ et $N := 7$. Le calcul numérique donne, avec `newton(initial)` :

$$\bar{u}(x) = 1.5454 \cos(x) + 0.1486 \cos(3x) + 0.0131 \cos(5x) + 0.0011 \cos(7x).$$

On a :

$$\det(J) \in [153704739.60219332, 153704739.60220668].$$

Ce qui justifie l'inversibilité de J . En fixant $r^* := 10^{-2}$ nous obtenons numériquement les bornes suivantes pour l'approximation $\bar{u}$:

$$Y \in [4.2320968914 \times 10^{-4}, 4.2320968916 \times 10^{-4}],$$

$$Z_1 \in [0.170327264655436, 0.170327264655438],$$

$$Z_2 \in [7.390644651319, 7.390644651321].$$

On définit le polynôme de rayon par :

$$p(r) := Y + (Z_1 - 1)r + \frac{1}{2}Z_2 r^2.$$

Le rayon de validation trouvé par l'algorithme est :

$$r \in [5.1199999999 \times 10^{-4}, 5.1200000001 \times 10^{-4}],$$

nous obtenons les inclusions rigoureuses suivantes :

$$p(r) \in [-6.1404478 \times 10^{-7}, -6.1404476 \times 10^{-7}],$$

ainsi que la vérification de la condition de contraction nécessaire à l'unicité :

$$Z_1 + Z_2 r \in [0.174111274716912, 0.174111274716914].$$

Ainsi, par le théorème 3, on en déduit qu'il existe une unique solution,

$$u^* \in \overline{B(\bar{u}, r)}$$

telle que :

$$F(u^*) = 0.$$

La représentation graphique de $\bar{u}$ donne :

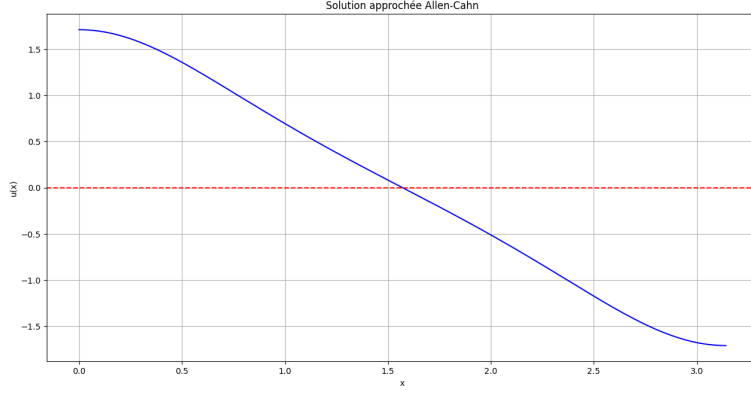


FIGURE 2 – Représentation graphique de $\bar{u}$

Conclusion

Ce travail a permis d'explorer les outils de validation numérique rigoureuse pour la recherche de zéros et d'équations différentielles. L'arithmétique d'intervalles a constitué le socle de notre approche, garantissant un contrôle strict des erreurs d'arrondi.

En s'appuyant sur le théorème de Newton–Kantorovich, nous avons montré comment transformer une approximation numérique en une preuve mathématique d'existence et d'unicité. Cette méthodologie a été illustrée avec succès sur l'équation d'Allen–Cahn en dimension infinie via une décomposition en séries de Fourier.

L'implémentation d'un opérateur hybride, alliant inversion matricielle pour les bas modes et analyse asymptotique pour la queue, a permis de traiter efficacement la dimension infinie. La prise en compte systématique du débordement spectral dans le calcul des bornes garantit que la certification obtenue est mathématiquement complète. Cette approche souligne la nécessité d'une synergie entre l'analyse fonctionnelle et le calcul certifié pour valider des solutions de modèles physiques complexes.

Ces résultats confirment que les preuves assistées par ordinateur offrent un cadre puissant pour l'étude rigoureuse des problèmes non linéaires.

Annexes

Notations

-Espaces ℓ_ν^1 pondérés : Étant donné $\nu \geq 1$, nous introduisons un espace de suites jouant un rôle central dans notre analyse :

$$\ell_\nu^1 := \left\{ u = (u_n)_{n \in \mathbb{N}} \in \mathbb{R}^{\mathbb{N}} \left| \|u\|_\nu := |u_0| + 2 \sum_{n=1}^{+\infty} |u_n| \nu^n < +\infty \right. \right\}.$$

Notons que pour tout $(u_n)_{n \in \mathbb{N}} \in \ell_\nu^1$, et pour tout $x \in [0, 2\pi]$, la fonction u construite par :

$$u(x) := u_0 + 2 \sum_{n=1}^{+\infty} u_n \cos(nx),$$

est bien définie par convergence uniforme.

En effet, pour tout $n \in \mathbb{N}^*$, on a :

$$|u_n \cos(nx)| \leq |u_n|.$$

Or $\nu \geq 1$, donc pour tout $n \in \mathbb{N}^*$:

$$|u_n| = |u_n| \nu^n \nu^{-n} \leq |u_n| \nu^n.$$

Comme :

$$\sum_{n=1}^{+\infty} |u_n| \nu^n < +\infty,$$

on en déduit, par domination, que la série définissant $u(x)$ converge uniformément sur $[0, 2\pi]$.

Ainsi, $u(x)$ est bien définie.

Nous désignons par ailleurs, $\mathcal{L}(\ell_\nu^1)$ comme étant l'ensemble des opérateurs linéaires continues de ℓ_ν^1 . On utilisera la même notation pour un espace de Banach $\mathcal{X}$ quelconque.

-Opérateurs de multiplication : Si l'espace de Banach $\mathcal{X}$ possède un opérateur de multiplication qui en fait une algèbre de Banach, alors ℓ_ν^1 est lui-même une algèbre de Banach, munie du produit de convolution discrète $*$ défini comme suit : pour tous $u, v \in \ell_\nu^1$,

$$(u * v)_n := \sum_{k \in \mathbb{Z}} u_{|k|} v_{|n-k|} \quad \forall n \in \mathbb{N}.$$

Quelques preuves complémentaires

Théorème A1 : Si u et v sont deux fonctions définies pour tout $x \in [0, 2\pi]$ par :

$$u(x) := u_0 + 2 \sum_{n=1}^{+\infty} u_n \cos(nx) \quad \text{et} \quad v(x) := v_0 + 2 \sum_{n=1}^{+\infty} v_n \cos(nx),$$

alors on a :

$$(uv)(x) = (u * v)_0 + 2 \sum_{n=1}^{+\infty} (u * v)_n \cos(nx),$$

Preuve : Soit $x \in [0, 2\pi]$. Notons que pour tout $n \in \mathbb{N}$:

$$\cos(nx) = \frac{e^{inx} + e^{-inx}}{2}.$$

Ainsi,

$$u(x) = u_0 + \sum_{n=1}^{+\infty} u_n e^{inx} + \sum_{n=1}^{+\infty} u_n e^{-inx} = \sum_{n \in \mathbb{Z}} u_{|n|} e^{inx}.$$

De même,

$$v(x) = \sum_{n \in \mathbb{Z}} v_{|n|} e^{inx}.$$

Ainsi,

$$(uv)(x) = \left(\sum_{n \in \mathbb{Z}} u_{|n|} e^{inx} \right) \left(\sum_{n \in \mathbb{Z}} v_{|n|} e^{inx} \right).$$

Par produit de Cauchy, cela donne :

$$(uv)(x) = \sum_{n \in \mathbb{Z}} (u * v)_n e^{inx},$$

Or on a immédiatement :

$$(u * v)_n = (u * v)_{|n|},$$

donc on peut réécrire :

$$(uv)(x) = (u * v)_0 + 2 \sum_{n=1}^{+\infty} (u * v)_n \cos(nx).$$

Ceci achève la preuve. □

Théorème A2 : Soit $(u, v) \in (\ell_\nu^1)^2$ alors :

$$\|u * v\|_\nu \leq \|u\|_\nu \|v\|_\nu.$$

Preuve : Soit $(u, v) \in (\ell_\nu^1)^2$. Nous avons :

$$\|u * v\|_\nu = |(u * v)_0| + 2 \sum_{n=1}^{+\infty} |(u * v)_n| \nu^n = \sum_{n \in \mathbb{Z}} |(u * v)_n| \nu^{|n|}.$$

Il vient ensuite que :

$$\|u * v\|_\nu \leq \sum_{n \in \mathbb{Z}} \sum_{k \in \mathbb{Z}} |u_k| |v_{n-k}| \nu^{|n|}.$$

Comme $\nu \geq 1$, on a l'inégalité $\nu^{|n|} \leq \nu^{|k|} \nu^{|n-k|}$. On en déduit :

$$\|u * v\|_\nu \leq \sum_{n \in \mathbb{Z}} \sum_{k \in \mathbb{Z}} \left(|u_k| \nu^{|k|} \right) \left(|v_{n-k}| \nu^{|n-k|} \right).$$

Par le théorème de Fubini, nous pouvons intervertir les sommes et effectuer le changement d'indice $j = n - k$:

$$\|u * v\|_\nu \leq \left(\sum_{k \in \mathbb{Z}} |u_k| \nu^{|k|} \right) \left(\sum_{j \in \mathbb{Z}} |v_j| \nu^{|j|} \right).$$

Or :

$$\left(\sum_{k \in \mathbb{Z}} |u_k| \nu^{|k|} \right) \left(\sum_{j \in \mathbb{Z}} |v_j| \nu^{|j|} \right) = \left(|u_0| + 2 \sum_{n=1}^{+\infty} |u_n| \nu^n \right) \left(|v_0| + 2 \sum_{n=1}^{+\infty} |v_n| \nu^n \right).$$

Enfin :

$$\left(|u_0| + 2 \sum_{n=1}^{+\infty} |u_n| \nu^n \right) \left(|v_0| + 2 \sum_{n=1}^{+\infty} |v_n| \nu^n \right) = \|u\|_\nu \|v\|_\nu.$$

Ceci termine la preuve. $\square$

Lemme A3 : Soit $M : \ell_\nu^1 \rightarrow \ell_\nu^1$ un opérateur linéaire représenté par une matrice infinie $(M_{ij})_{i,j \geq 0}$. Alors la norme de cet opérateur, définie par :

$$\|M\|_{\mathcal{L}(\ell_\nu^1)} = \sup_{u \neq 0} \frac{\|Mu\|_\nu}{\|u\|_\nu}$$

est donnée par la formule :

$$\|M\|_{\mathcal{L}(\ell_\nu^1)} = \sup_{j \geq 0} \frac{1}{\omega_j} \sum_{i=0}^{+\infty} |M_{ij}| \omega_i$$

Preuve : Soit $u \in \ell_\nu^1$. On écrit $\|u\|_\nu$ de la manière suivante :

$$\|u\|_\nu = \sum_{n=0}^{+\infty} |u_n| \omega_n$$

où les poids sont définis par $\omega_0 = 1$ et $\omega_n = 2\nu^n$ pour $n \geq 1$.

Notons $v := Mu$. Par définition, la i -ème composante de v est :

$$v_i = \sum_{j=0}^{+\infty} M_{ij} u_j.$$

En utilisant l'inégalité triangulaire, nous majorons chaque terme $|v_i|$:

$$\|v\|_\nu = \|Mu\|_\nu \leq \sum_{i=0}^{+\infty} \left(\sum_{j=0}^{+\infty} |M_{ij}| |u_j| \right) \omega_i$$

Par le théorème de Fubini pour les séries à termes positifs :

$$\|Mu\|_\nu \leq \sum_{j=0}^{+\infty} |u_j| \left(\sum_{i=0}^{+\infty} |M_{ij}| \omega_i \right)$$

Pour faire apparaître la définition de $\|u\|_\nu$, nous multiplions et divisons par $\omega_j > 0$:

$$\|Mu\|_\nu \leq \sum_{j=0}^{+\infty} (|u_j| \omega_j) \left(\frac{1}{\omega_j} \sum_{i=0}^{+\infty} |M_{ij}| \omega_i \right)$$

Soit $S_j := \frac{1}{\omega_j} \sum_{i=0}^{+\infty} |M_{ij}| \omega_i$. En posant $C := \sup_{j \geq 0} S_j$, nous avons :

$$\|Mu\|_\nu \leq C \sum_{j=0}^{+\infty} |u_j| \omega_j = C \|u\|_\nu$$

D'où :

$$\frac{\|Mu\|_\nu}{\|u\|_\nu} \leq \sup_{j \geq 0} \frac{1}{\omega_j} \sum_{i=0}^{+\infty} |M_{ij}| \omega_i$$

Pour prouver que la borne est exactement égale à ce supremum, considérons un indice j_0 fixé. Construisons un vecteur particulier $e^{(j_0)}$ tel que toutes ses composantes soient nulles sauf celle d'indice j_0 . Soit $u = e^{(j_0)}$, avec $u_j = \delta_{j,j_0}$. La norme de ce vecteur est :

$$\|u\|_\nu = \sum_{j=0}^{+\infty} \delta_{j,j_0} \omega_j = \omega_{j_0}$$

Le vecteur image $v = Mu$ correspond à la j_0 -ème colonne de la matrice M , c'est-à-dire $v_i = M_{i,j_0}$. Sa norme est :

$$\|Mu\|_\nu = \sum_{i=0}^{+\infty} |M_{i,j_0}| \omega_i$$

Le rapport des normes pour ce vecteur particulier est :

$$\frac{\|Mu\|_\nu}{\|u\|_\nu} = \frac{\sum_{i=0}^{+\infty} |M_{i,j_0}| \omega_i}{\omega_{j_0}} = S_{j_0}$$

On a donc :

$$\|M\|_{\mathcal{L}(\ell_\nu^1)} \geq \frac{\|Mu\|_\nu}{\|u\|_\nu} = S_{j_0}$$

Puisque cette inégalité est vraie pour tout $j_0 \geq 0$, elle est vraie pour le supremum sur j :

$$\|M\|_{\mathcal{L}(\ell_\nu^1)} \geq \sup_{j \geq 0} S_j$$

En combinant les résultats nous concluons :

$$\|M\|_{\mathcal{L}(\ell_\nu^1)} = \sup_{j \geq 0} \frac{1}{\omega_j} \sum_{i=0}^{+\infty} |M_{ij}| \omega_i$$

La preuve se termine ici.

□

Codes Python

```
from flint import arb, arb_mat
import numpy as np

# =====
# PARAMETRES
# =====

N = 7
M_ext = 3 * N # On étend jusqu'à 3N pour capturer la convolution
nu = arb("1.1")

# =====
# POIDS ET NORME (Etendus jusqu'à 3N)
# =====

weights = [arb(1)]
for n in range(1, M_ext + 1):
    weights.append(2 * (nu**n))

def norm_nu(a):
    s = abs(a[0])
    for n in range(1, len(a)):
        s += 2 * abs(a[n]) * (nu**n)
    return s

# =====
# CONVOLUTION ET OPERATEURS
# =====

def convolution_extended(a, b, target_size):
    out = [arb(0) for _ in range(target_size)]
    len_a, len_b = len(a), len(b)
    for n in range(target_size):
        s = arb(0)
        for k in range(len_a):
            # Termes de type a_k * b_{|n-k|}
            j1 = abs(n - k)
            if j1 < len_b:
                s += a[k] * b[j1]
            # Termes de type a_k * b_{n+k}
            j2 = n + k
            if k != 0 and j2 < len_b:
                s += a[k] * b[j2]
        out[n] = s
    return out
```

Listing 1 : Outils pour l'arithmétique rigoureuse dans ℓ_ν^1

```

def DF_matrix_extended(a_bar):

    # aa = u_bar * u_bar (taille 2N+1)
    aa = convolution_extended(a_bar, a_bar, 2 * N + 1)

    J = arb_mat(M_ext + 1, N + 1)
    for j in range(N + 1):
        e_j = [arb(0) for _ in range(N + 1)]
        e_j[j] = arb(1)

        # 3 * u_bar^2 * e_j (taille 3N+1)
        three_aah = [3 * x for x in convolution_extended(aa, e_j,
            M_ext + 1)]

        for i in range(M_ext + 1):
            val = arb(0)
            if i == j: # Terme L = n^2 + 1
                val += arb(i * i + 1)
            val -= three_aah[i]
            J[i, j] = val
    return J

# =====
# NEWTON (Flottant pour l'approximation)
# =====

def F_float(a):
    # Version simplifiée pour Newton (midpoints)
    N_loc = len(a) - 1
    a_arb = [arb(x) for x in a]
    aa = convolution_extended(a_arb, a_arb, N_loc + 1)
    aaa = convolution_extended(aa, a_arb, N_loc + 1)
    return [float((arb(n*n+1)*a_arb[n] - aaa[n]).mid()) for n in
        range(N_loc+1)]

def newton(initial):
    a = np.array(initial, dtype=float)
    for _ in range(15):
        a_arb = [arb(x) for x in a]
        Fv = np.array(F_float(a))
        # Jacobienne locale N * N pour l'inversion de Newton
        J_small = arb_mat(N+1, N+1)
        aa = convolution_extended(a_arb, a_arb, N+1)
        for j in range(N+1):
            e = [arb(0) for _ in range(N+1)]; e[j] = arb(1)
            col = [3*x for x in convolution_extended(aa, e, N+1)]
            for i in range(N+1):
                J_small[i, j] = arb(i*i+1) if i==j else arb(0)
                J_small[i, j] -= col[i]

        Jn = np.array([[float(J_small[i, j].mid()) for j in range(N
            +1)] for i in range(N+1)])
        a = a - np.linalg.solve(Jn, Fv)
    return a

```

Listing 2 : Définition de l'opérateur d'Allen-Cahn et de sa différentielle


```

# Execution Newton
a_num = newton([0, 1.3, 0, 0.12, 0, 0.015, 0, 0.002])
a_bar = [arb(x) for x in a_num]

print(a_bar)

# =====
# CALCUL DES BORNES RIGOUREUSES
# =====

# 1. J_ext (3N+1 x N+1) et R (N+1 x N+1)
J_ext = DF_matrix_extended(a_bar)
# On extrait le bloc N+1 * N+1 pour calculer l'inverse R
J_N = arb_mat(N+1, N+1)
for i in range(N+1):
    for j in range(N+1):
        J_N[i, j] = J_ext[i, j]
R = J_N.inv()

# 2. Construction de B = I - A*DF(u_bar) de taille (3N+1 * N+1)
B = arb_mat(M_ext + 1, N + 1)
for j in range(N + 1):
    # Calcul de la colonne j : B*e_j = e_j - A*(J_ext * e_j)
    # Soit col = J_ext[:, j]
    col_j = [J_ext[i, j] for i in range(M_ext + 1)]

    # Application de A :
    # - Sur les modes i <= N : R * col_j[0:N+1]
    # - Sur les modes i > N : col_j[i] / (i^2 + 1)
    A_J = [arb(0) for _ in range(M_ext + 1)]

    # Partie finie
    vec_low = arb_mat(N+1, 1)
    for i in range(N+1): vec_low[i,0] = col_j[i]
    res_low = R * vec_low
    for i in range(N+1): A_J[i] = res_low[i,0]

    # Partie queue (n > N)
    for i in range(N+1, M_ext + 1):
        A_J[i] = col_j[i] / arb(i*i + 1)

    # B = I - A_J
    for i in range(M_ext + 1):
        val = arb(1) if i == j else arb(0)
        B[i, j] = val - A_J[i]

# 3. Z1_finite
Z1_finite = arb(0)
for j in range(N + 1):
    s = arb(0)
    for i in range(M_ext + 1):
        s += abs(B[i, j]) * weights[i]
    s /= weights[j]
    Z1_finite = max(Z1_finite, s)

# 4. Y (Residu)
# u_bar est dans X_N, donc F(u_bar) a des termes jusqu'à 3N

```

```

aa_res = convolution_extended(a_bar, a_bar, 2*N+1)
aaa_res = convolution_extended(aa_res, a_bar, M_ext+1)
F_u_bar = [arb(n*n+1)*a_bar[n] - aaa_res[n] if n <= N else -aaa_res
            [n] for n in range(M_ext+1)]

# Application de A à F(u_bar)
AF_u_bar = [arb(0) for _ in range(M_ext+1)]
vec_F = arb_mat(N+1, 1)
for i in range(N+1): vec_F[i,0] = F_u_bar[i]
res_R = R * vec_F
for i in range(N+1): AF_u_bar[i] = res_R[i,0]
for i in range(N+1, M_ext+1): AF_u_bar[i] = F_u_bar[i] / arb(i*i+1)

Y = norm_nu(AF_u_bar)

# 5. Z1_queue et Norme de A
tail_factor = arb(1) / arb((N + 1) ** 2 + 1)
u_norm = norm_nu(a_bar)
Z1_tail = 3 * (u_norm**2) * tail_factor
Z1 = max(Z1_finite, Z1_tail)

# Norme A
A_norm_finite = arb(0)
for j in range(N + 1):
    s = arb(0)
    for i in range(N + 1): s += abs(R[i, j]) * weights[i]
    s /= weights[j]
    A_norm_finite = max(A_norm_finite, s)
A_norm = max(A_norm_finite, tail_factor)

# Z2 et Validation
r_star = arb("1e-2")
Z2 = 3 * A_norm * (2*u_norm + r_star)

def p(r): return Y + (Z1 - 1) * r + arb("0.5") * Z2 * r * r

```

Listing 3 : Calcul des bornes

```

# =====
# VALIDATION FINALE
# =====

validated = False
steps_check = 5000

for k in range(1, steps_check + 1):
    r = arb(k) * r_star / arb(steps_check)

    # Condition :  $Y + (Z1-1)r + 0.5*Z2*r^2 < 0$ 
    if p(r).upper() < 0:
        validated = True
        contraction_value = Z1 + Z2 * r

        print("\n=====")
        print("          PREUVE VALIDÉE          ")
        print("=====\\n")
        print(f"Rayon r trouvé : {r}")
        print(f"Y (Résidu)      : {Y}")
        print(f"Z1              : {Z1}")
        print(f"Z2              : {Z2}")
        print("-" * 32)
        print(f"p(r)            : {p(r)}")
        print(f"Z1 + Z2*r      : {contraction_value}")
        print("-" * 32)
        print("Les conditions du théorème de Newton-Kantorovich")
        print("sont satisfaites de manière rigoureuse.")
        break

if not validated:
    print("\n=====")
    print("          ÉCHEC DE LA VALIDATION          ")
    print("=====")
    print("p(r) ne possède pas de valeur négative.")
    print("Vérifiez N, nu ou l'approximation initiale.")

```

Listing 4 : Validation des résultats

Références

- [1] Maxime Breden. *Computer-assisted proofs for differential equations and dynamical systems*. Analysis of PDEs [math.AP]. Institut Polytechnique de Paris, 2025. *tel-05142432v2*.